

20 Essential MERN Stack Interview Questions

with PostgreSQL (Not MongoDB)

Complete Guide with Answers & Code Examples

Generated on: June 21, 2026

Table of Contents

- REACT (Frontend) - 7 Questions
- NODE.JS & EXPRESS - 6 Questions
- POSTGRESQL - 4 Questions (Different from MongoDB!)
- FULL STACK & INTEGRATION - 3 Questions

REACT (Frontend) - 7 Questions

1. What is the Virtual DOM and how does it improve performance?

The Virtual DOM is a lightweight JavaScript representation of the actual DOM. React maintains a Virtual DOM in memory and updates only the parts of the real DOM that have changed.

2. Explain the difference between State and Props

State is mutable data managed by component; Props are immutable data passed from parent.

3. What are React Hooks? Explain useState and useEffect

Hooks are functions that let you use state and other React features in functional components.

4. What is the difference between Controlled and Uncontrolled Components?

Controlled: React state controls input. Uncontrolled: DOM manages input value using refs.

5. Explain the React Component Lifecycle (Class vs Functional)

Lifecycle Phases: Mount → Update → Unmount. Class uses methods; Functional uses Hooks.

6. What are React Context and useContext?

Context API allows passing data through component tree without prop drilling.

7. What is the difference between shallow and deep copying in React?

Shallow: Only copies first level. Deep: Creates completely independent copy.

NODE.JS & EXPRESS - 6 Questions

8. What is middleware in Express?

Middleware functions execute in request-response cycle with access to req, res, and next. Order matters!

9. Explain REST API principles and HTTP methods

GET (retrieve), POST (create), PUT (full update), PATCH (partial), DELETE (remove).

10. What is async/await and how is it different from Promises?

Async/await is cleaner syntax. Better error handling with try/catch. More maintainable.

11. How do you handle errors in Express?

Use try-catch in async routes. Pass errors to error middleware with next(error). Must have 4 parameters.

12. What is JWT (JSON Web Token)?

Token-based authentication. Server issues token after login; client sends it with requests.

POSTGRESQL - 4 Questions

■ **IMPORTANT: These are different from MongoDB!**

13. What are Tables and Records in PostgreSQL?

PostgreSQL stores data in **tables** (with defined schemas) containing **records** (rows with structured data). Unlike MongoDB's flexible schema, PostgreSQL requires predefined table structure.

Key Differences from MongoDB:

- Fixed schema (must define columns upfront)
- Relational data with foreign keys
- ACID compliance by default
- Uses SQL for queries
- Parameterized queries (\$1, \$2) prevent SQL injection

Basic Operations:

- INSERT: Add new records
- SELECT: Retrieve records
- UPDATE: Modify records
- DELETE: Remove records

14. What is an ORM? Explain Sequelize or TypeORM

An ORM (Object-Relational Mapping) is a tool that maps database tables to JavaScript classes/objects, allowing you to interact with PostgreSQL using OOP concepts.

Sequelize (Most popular for PostgreSQL):

- Define models that map to tables
- Type-safe operations
- Automatic validation
- Handles relationships (associations)
- Migrations for schema changes

Advantages of ORM:

- Write less SQL
- Better code organization
- Easier to maintain
- Database agnostic (switch DB easily)

15. What are Indexes in PostgreSQL?

Indexes improve query performance by allowing PostgreSQL to find data without scanning all records. Similar to book indices that help find topics quickly.

Index Types:

- Single-column: Fast lookup on one field (email, id)
- Composite: Multiple columns (age, email)
- Unique: Ensures uniqueness (email)
- Full-text: Search in text content

Performance Impact:

- Without index: Scan all 1M records (slow - $O(n)$)

- With index: Find instantly using B-tree (fast - $O(\log n)$)
- Trade-off: Faster SELECT, slower INSERT/UPDATE/DELETE

16. What are Relationships in Databases?

Relationships define how tables relate to each other. PostgreSQL enforces relationships using foreign keys.

Types:

- **One-to-Many (1:N)**: User has many posts (user_id in posts)
- **Many-to-Many (N:N)**: Users and courses (junction table)
- **One-to-One (1:1)**: User has one profile (foreign key with unique constraint)

vs MongoDB: MongoDB has no foreign keys. Uses embedded documents or manual joins.

FULL STACK & INTEGRATION - 3 Questions

17. How do you connect React frontend to Express + PostgreSQL?

Backend Setup:

1. Create PostgreSQL connection pool
2. Define Express routes with SQL queries
3. Use parameterized queries (\$1, \$2) to prevent SQL injection
4. Enable CORS for frontend communication
5. Return JSON responses

Frontend Setup:

1. Use useEffect to fetch data
2. Use fetch() or axios for HTTP calls
3. Update state with response data
4. Handle loading and error states

18. What are environment variables and why important?

Store sensitive data (database URL, JWT secret, API keys) outside code in .env file.

In PostgreSQL context:

DATABASE_URL=postgresql://user:pass@host:5432/dbname
Use this to create connection pool without hardcoding credentials.

19. How do you deploy MERN with PostgreSQL?

Key difference from MongoDB:

- MongoDB: Cloud provider (Atlas)
- PostgreSQL: Managed service (AWS RDS, Railway, Supabase)

Deployment Steps:

1. Build React app
2. Setup PostgreSQL on managed service
3. Set DATABASE_URL environment variable
4. Run database migrations
5. Deploy backend + serve React build

20. PostgreSQL-specific optimization techniques

Query Optimization:

- Use EXPLAIN ANALYZE to understand query performance
- Create indexes on frequently filtered columns
- Avoid N+1 problem: Use JOINS instead of loop queries
- Batch operations: INSERT multiple rows at once

Connection Pooling:

Use pool instead of creating new connection per request. Max 20-30 connections.

PostgreSQL vs MongoDB Quick Reference

Feature	PostgreSQL	MongoDB
Schema	Fixed (predefined)	Flexible (dynamic)
Query Language	SQL	JavaScript/BSON
Relationships	Foreign keys (enforced)	Manual or embedded
ACID	Always	Recent versions
Scaling	Vertical	Horizontal (sharding)
ORM	Sequelize, TypeORM	Mongoose (ODM)
Data Size	Great for structured	Great for unstructured
Complex Queries	JOINS (powerful)	Aggregation pipeline

Quick Reference Checklist

- ✓ Virtual DOM & Reconciliation
- ✓ State vs Props
- ✓ Hooks (useState, useEffect, useContext)
- ✓ HTTP Methods & REST API
- ✓ Async/Await & Promises
- ✓ Middleware in Express
- ✓ JWT Authentication
- ✓ PostgreSQL Tables & SQL Queries
- ✓ ORM Concepts (Sequelize/TypeORM)
- ✓ Database Indexes
- ✓ Relationships (1:N, N:N, 1:1)
- ✓ Connection Pooling
- ✓ N+1 Query Problem
- ✓ Error Handling
- ✓ CORS & Environment Variables
- ✓ Deployment with PostgreSQL

Good luck with your interview! Your PostgreSQL experience is a strong asset. Explain your answers clearly with examples from your actual projects.